

Ejercicio 3

Tabla de atributos

#	Tipo de dato	Nombre	Visibilidad	Motivo	Clase
1	int	numeroOrden	private	Almacena un número único que identificará cada orden de servicio.	Orden
2	String	nombrePropietario	private	Almacena el nombre del propietario del vehículo.	Orden
3	String	placa	private	Almacena la placa del vehículo.	Orden
4	String	descripcionServicio	private	Almacena la descripción del servicio que se le realizará al vehículo.	Orden
5	double	costoEstimado	private	Almacena el costo estimado del servicio.	Orden
6	List<Orden>	ordenes	private	Almacena las ordenes de servicio activas mediante un arreglo dinámico.	GestionOrdenes

7	Scanner	scanner	private	Permite recibir datos del usuario.	OrdenView
8	GestionOrdenes	gestionOrdenes	private	Le proporciona al controlador acceso a las operaciones del modelo.	OrdenController
9	OrdenView	vista	private	Permite la comunicación entre vista y controlador.	OrdenController

Tabla de métodos

#	Tipo	Nombre	Visibilidad	Motivo	Parámetros	Clase
1	Constructor	Orden	public	Inicializa una orden con todos sus datos.	int numeroOrden, String nombrePropietario, String placa, String descripcionServicio, double costoEstimado	Orden
2	int	getNumeroOrden	public	Devuelve el número de la orden.		Orden
3	String	getNombrePropietario	public	Devuelve el nombre del propietario.		Orden

4	String	getPlaca	public	Devuelve la placa del vehículo.		Orden
5	String	getDescripcionServicio	public	Devuelve la descripción del servicio.		Orden
6	void	setDescripcionServicio	public	Modifica la descripción del servicio.	String descripcionServicio	Orden
7	double	getCostoEstimado	public	Devuelve el costo estimado de la orden.		Orden
8	void	setCostoEstimado	public	Modifica el costo estimado de la orden.	double costoEstimado	Orden
9	String	mostrarInformacion	public	Devuelve la información completa de la orden en formato de texto.		Orden
10	Constructor	GestionOrdenes	public	Inicia el arreglo dinámico.		GestionOrdenes
11	void	registrarOrden	public	Valida y agrega una orden al arreglo.	Orden orden	GestionOrdenes

12	List<Orden >	obtenerOrdenes	public	Devuelve todas las órdenes almacenadas.		GestionOrdenes
13	Orden	buscarPorNumero	public	Busca y devuelve una orden mediante su número.	int numeroOrden	GestionOrdenes
14	void	modificarOrden	public	Modifica la descripción y el costo de una orden existente.	int numeroOrden, String nuevaDescripcion, double nuevoCosto	GestionOrdenes
15	void	cancelarOrden	public	Elimina de la colección la orden indicada.	int numeroOrden	GestionOrdenes
16	ArrayList<Orden>	buscarPorPlaca	public	Devuelve todas las órdenes asociadas a una placa.	String placa	GestionOrdenes
17	double	calcularCostoTotal	public	Suma los costos estimados de las órdenes activas.		GestionOrdenes
18	double	calcularCostoPromedio	public	Calcula el costo		GestionOrdenes

				promedio de las órdenes activas.		
19	Orden	obtenerOrdenMayorCosto	public	Devuelve la orden que posee el costo estimado más alto.		GestionOrdenes
20	int	obtenerCantidadOrdenes	public	Devuelve la cantidad actual de órdenes registradas.		GestionOrdenes
21	Constructor	OrdenView	public	Inicializa el objeto Scanner utilizado para leer datos.		OrdenView
22	void	mostrarMenu	public	Muestra las opciones del menú principal.		OrdenView
23	String	solicitarDato	public	Muestra una indicación y devuelve el dato ingresado por el usuario.	String mensaje	OrdenView
24	void	mostrarMensaje	public	Muestra mensajes	String mensaje	OrdenView

				informativos, confirmaciones o errores.		
25	void	mostrarOrden	public	Muestra la información de una orden.	Orden orden	OrdenView
26	void	mostrarOrdenes	public	Muestra todas las órdenes contenidas en una lista.	List<Orden> ordenes	OrdenView
27	void	mostrarReporteCostos	public	Muestra el costo total y el costo promedio.	double total, double promedio	OrdenView
28	Constructor	OrdenController	public	Inicializa el controlador con el modelo y la vista.	GestionOrdenes gestionOrdenes, OrdenView vista	OrdenController
29	void	iniciar	public	Inicia el menú y mantiene el programa en ejecución.		OrdenController
30	void	registrarOrden	private	Solicita los datos y coordina el registro de una orden.		OrdenController

31	void	consultarOrdenes	private	Obtiene las órdenes y solicita a la vista que las muestre.		OrdenController
32	void	buscarOrden	private	Solicita el número y coordina la búsqueda de una orden.		OrdenController
33	void	modificarOrden	private	Solicita los nuevos datos y coordina la modificación de una orden.		OrdenController
34	void	cancelarOrden	private	Solicita el número y coordina la eliminación de una orden.		OrdenController
35	void	consultarOrdenes PorPlaca	private	Solicita una placa y muestra todas sus órdenes asociadas.		OrdenController
36	void	mostrarReporteCostos	private	Obtiene del modelo el total y el promedio para mostrarlos.		OrdenController

37	void	mostrarOrdenMayorCosto	private	Obtiene y muestra la orden con el costo más alto.		OrdenController
38	void	mostrarCantidadOrdenes	private	Obtiene y muestra la cantidad de órdenes registradas.		OrdenController
39	void	main	public static	Inicia la ejecución del programa	String[] args	Main

1. ¿Qué clases son necesarias para representar el problema planteado? Siguiendo el modelo MVC, plantee las siguientes clases: Orden, GestionOrdenes, OrdenView, OrdenController y Main.

La clase Orden representa una orden de servicio; GestionOrdenes administra la colección dinámica; OrdenView gestiona la interacción con el usuario; OrdenController coordina el modelo y la vista; y Main crea las instancias e inicia el programa.

2. ¿Qué propiedades y métodos tendrá cada clase? Ver tablas.
3. ¿Qué tipo deben tener las propiedades y métodos de cada clase? Ver tablas.
4. ¿Qué información deberá almacenarse dentro de la colección dinámica? La información que guardará la colección dinámica serán los objetos de la clase Orden. Por lo que, cada objeto contendrá: el número de orden, el nombre de propietario, la placa del vehículo, la descripción del servicio y el costo estimado.
5. ¿Dónde utilizará List y dónde realizará la creación mediante ArrayList? En la clase GestionOrdenes, la propiedad se declarará utilizando la interfaz List y la creación de la colección se hará en el constructor de la clase mediante ArrayList.
6. ¿Cuáles deben ser los modificadores de visibilidad de los miembros de cada clase? Ver tablas.

7. ¿Qué parámetros serán requeridos por los métodos de sus clases? Ver tablas.
8. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores iniciales les asignará?
Los objetos de la clase Orden recibirán sus valores iniciales mediante el constructor, al cual se enviarán el número de orden, el nombre del propietario, la placa del vehículo, la descripción del servicio y el costo estimado ingresados por el usuario. La colección de GestionOrdenes iniciará como un ArrayList vacío y OrdenView inicializará el objeto Scanner.
9. ¿Cómo identificará una orden específica dentro de la colección? Cada orden será identificada por su número único. La colección se recorrerá hasta encontrar una orden cuyo numeroOrden coincida con el número solicitado.
10. ¿Cómo agregará y eliminará órdenes de la colección dinámica? Las órdenes se agregarán con add() después de validar sus datos. Para eliminarlas, se buscarán por su número y se utilizará remove().
11. ¿Cómo realizará la búsqueda de todas las órdenes asociadas a una misma placa? Se recorrerá la colección y se comparará cada placa con la solicitada mediante equalsIgnoreCase(). Las coincidencias se guardarán en una nueva lista.
12. ¿Cómo recorrerá la colección para calcular el valor total y el costo promedio de las órdenes? Se utilizará un ciclo for-each para sumar los costos. El promedio se calculará dividiendo el total entre la cantidad de órdenes, siempre que la colección no esté vacía.
13. ¿Cómo determinará cuál es la orden con el costo estimado más alto? Se recorrerá la colección mediante un ciclo for-each, comparando los costos y guardando la orden que tenga el costo estimado más alto.
14. ¿Qué situaciones dentro del programa pueden producir una excepción? Pueden ocurrir excepciones al ingresar datos numéricos incorrectos, utilizar datos vacíos, repetir un número de orden o intentar buscar, modificar o eliminar una orden inexistente.
15. ¿Qué operaciones serán protegidas utilizando try-catch? Se protegerán las conversiones numéricas, el registro de datos inválidos y las operaciones de búsqueda, modificación o cancelación de órdenes inexistentes.
16. ¿Qué acción realizará mediante el bloque finally y por qué debe ejecutarse independientemente del resultado de la operación? El bloque finally mostrará que la operación terminó y que se regresará al menú. Se ejecutará tanto si la operación fue exitosa como si ocurrió una excepción.